

Project 2

Problems

- Memory Leaks caused by:
 - Improper array allocation in:
 - Edge*** G in Graph.h
 - Vertex* list[numOfVertices+1] in Main.cpp
 - Array Indexing
 - int* arrOdd in Main.cpp

Bugs

- while(!cin.eof()) may cause additional cin for start and end vertice, $i < \text{numOfEdges}$ should alleviate this issue but not guaranteed to

External books, and/or websites used or referenced (APA 7)

Programiz. (n.d.). *Priority queue in data structure (with C, C++ and Java examples)*. Programiz. <https://www.programiz.com/dsa/priority-queue>

W3Schools. (n.d.). *Dijkstra's algorithm*. W3Schools. https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php

W3Schools. (n.d.). *Graphs in data structure*. W3Schools. https://www.w3schools.com/dsa/dsa_theory_graphs.php

cplusplus.com. (n.d.). *std::setw*. <http://cplusplus.com/reference/iomanip/setw/>

WilliamFiset. (2022, February 1). *Dijkstra's shortest path algorithm – Graph theory 08* [Video]. YouTube. https://youtu.be/bZkzH5x0SKU?si=bJEwmyOaHC_xYPx7

Design Decisions

I chose to represent the graph using an adjacency matrix, which is a development-efficient and flexible way to handle sparse graphs. The graph is a 2d array of `Edge*`, which was used because of the more significant roles of Edge objects in the context of the program. It also makes it more compatible with weighted graphs. I stayed with the general design of classes. However, the Priority Queue uses Node structs. In addition, the Queue was a big player in determining the shortest path from Dijkstra's Algorithm.